

**Московский авиационный институт
(национальный исследовательский университет)**

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Информационный поиск»

Студент: М.А. Пирязев
Преподаватель: А.А. Кухтичев
Группа: М8О-407Б-22
Дата: 30.10.2025
Оценка:
Подпись:

Москва, 2025

Оглавление

Лабораторная работа №1 «Добыча корпуса документов».....	2
Описание	3
Сведения о корпусе	3
Исходный код.....	4
Выводы	6
Список литературы.....	7

Лабораторная работа №1 «Добыча корпуса документов»

Необходимо подготовить корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать его к себе на компьютер. В отчёте нужно указать источник данных.
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Разбить на документы.
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер «сырых» данных.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

Описание

Целью лабораторной работы является подготовка корпуса документов для его последующего использования в информационном поиске. В ходе работы требуется загрузить данные из открытых источников, провести их предварительную обработку, извлечь содержательную часть и получить статистические характеристики собранного корпуса.

В качестве источников данных были выбраны русская Википедия и русский Stack Overflow. Выбор этих источников обусловлен их доступностью в виде открытых дампов, достаточным объёмом содержательного материала и высокой качеством разметки. Оба источника содержат хорошо структурированный текст с метаинформацией, которая может быть полезна для анализа и поиска.

Русская Википедия предоставляет энциклопедические статьи по различным темам, что позволяет работать с разнообразным контентом на русском языке. Stack Overflow, в свою очередь, содержит вопросы и ответы по программированию, предоставляя материал, который часто содержит код и технические описания.

Для загрузки и обработки данных был разработан программный комплекс, состоящий из нескольких модулей. Каждый модуль отвечает за отдельный этап конвейера обработки: загрузку дампов, потоковый парсинг XML, очистку текста от разметки и агрегирование статистики.

Сведения о корпусе

Разработанная система позволила собрать корпус, состоящий из двух источников. Размер сырых данных Википедии составляет примерно 3,5 гигабайта в сжатом формате (bz2), Stack Overflow — около 400–600 мегабайт в формате 7z. После обработки и извлечения текста были получены файлы JSONL, содержащие структурированные документы с метаинформацией.

Каждый документ в корпусе содержит следующие поля: уникальный идентификатор, внешний идентификатор из исходного источника, название источника, заголовок документа, очищенный текст и метаданные (URL, дата создания, теги и другие атрибуты). Минимальная длина текста документа установлена в 100 символов для Википедии и 50 символов для Stack Overflow, что позволяет отсеять очень короткие записи.

Исходный код

Архитектура решения базируется на модульном подходе с использованием потокового парсинга для эффективной работы с большими объёмами данных. Основным вызовом при работе с дампами Википедии размером 3–4 гигабайта было ограничение памяти, поэтому был выбран подход с использованием `iterparse` из стандартной библиотеки `xml.etree.ElementTree`, который позволяет обрабатывать XML-документ без загрузки его целиком в оперативную память.

Конфигурационная система использует переменные окружения из файла `.env`, что позволяет управлять параметрами работы программы без изменения исходного кода. К основным параметрам относятся пути к директориям корпуса, минимальные количества документов для каждого источника и целевой размер корпуса.

Для обработки Википедии был разработан парсер `WikiParser`, который выполняет следующие операции. Во-первых, он автоматически определяет XML-namespace из корневого элемента документа. Затем происходит потоковая обработка каждой страницы: извлекаются заголовок, идентификатор страницы и текст последней версии. На этапе фильтрации отсекаются служебные страницы (начинающиеся с префиксов вроде «Википедия:», «Файл:», «Категория:» и т. д.), редиректы и страницы с недостаточным объёмом текста.

Функция `clean_wikitext` осуществляет очистку текста от разметки Вики, удаляя шаблоны в формате `{{...}}`, ссылки на файлы, преобразуя ссылки в простой текст и

убирая HTML-теги. Нормализация пробелов приводит множественные пробелы и переводы строк к стандартному виду.

Парсер Stack Exchange, реализованный в классе StackExchangeParser, работает со структурой XML дампов, где каждая строка (<row>) представляет собой либо вопрос (PostTypeId=1), либо ответ (PostTypeId=2). В отчёт включаются только вопросы, так как они содержат содержательно полные формулировки. Извлекаются текст, заголовок, оценка (score), количество просмотров и теги. Функция clean_html удаляет HTML-теги, но оставляет разметку блоков кода, которые заменяются на текстовые метки для сохранения информации о структуре кода.

Вспомогательные скрипты для загрузки дампов (download_wiki.py и download_stackexchange.py) используют стандартный urllib для загрузки файлов с отображением прогресса. Разработана система тестирования с использованием pytest, которая проверяет корректность очистки текста, фильтрации документов и соответствие структуры выходных данных.

Выводы

Выполнив первую лабораторную работу по курсу «Информационный поиск», я приобрёл практические навыки работы с большими объёмами неструктурированных данных и их предварительной обработки. В процессе разработки я столкнулся с различными вызовами, такими как эффективное использование памяти при обработке многогигабайтных файлов и корректная работа с XML-пространствами имён в Python.

Основной практический результат — успешно собран корпус из русской Википедии и Stack Overflow с применением потокового парсинга, что позволяет работать с данными объёмом более 3 гигабайт без значительного потребления оперативной памяти. Я узнал, как правильно структурировать код для работы с внешними API и локальными дампами, использовать регулярные выражения для очистки текста от разметки разных форматов и применять тестирование для проверки корректности преобразований данных.

На практике понимаю, что для реальных систем информационного поиска критично иметь надёжный конвейер обработки данных, который обеспечивает консистентность и качество входящих документов. Применённые навыки будут полезны при работе с другими типами текстовых корпусов и при разработке систем, требующих обработки больших объёмов данных. Кроме того, опыт организации кода, конфигурирования и тестирования является основополагающим для профессиональной разработки.

Список литературы

- [1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д.А. Ключина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))
- [2] ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018.
- [3] Воронцов, К. В. Разведочный информационный поиск (лекция/видеоматериал)